CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 07
Dynamic Programming II — Optimization & Pseudo-Polynomial Algorithms

Better states, faster transitions, less memory, and the moment a "polynomial" DP turns out to be exponential.

Saturday 7 November 2026 · 3 hours

PS2 due today · Midterm (L01–L07) working date Sat 14 Nov

Motivating problem: same problem, different units

A scheduler packs 100 jobs onto a server to maximise priority value, subject to memory capacity.

Memory measured in…	capacity W	$O(nW)$ DP work
GB	256	2.6×10^4
MB	262,144	2.6×10^7
bytes	2.7×10^{11}	2.7×10^{13}

The input grew by a few *characters*. The running time grew by a factor of 10^9.

DISCUSS · 3 MIN

Is the $O(nW)$ knapsack DP a polynomial-time algorithm? What is the size of the input "$W = 274877906944$"?

What would you do if the job sizes really are only known to the byte?

BLOCK A · 00:10–01:00

Harder state modelling

Weighted intervals · LIS in $O(n \log n)$ · transition and state reduction

Weighted interval scheduling

Intervals $j = 1..n$ sorted by finish time, with values v_j. Choose compatible intervals of maximum total value.

$p(j)$ = the largest index $i < j$ with $f_i \leq s_j$ (0 if none): computable by binary search.

$$OPT(j) = \max\big(OPT(j-1),\ v_j + OPT(p(j))\big), \quad OPT(0) = 0.$$

Last decision: is interval j (the last to finish) in the solution? If yes, the rest is an optimal solution among intervals compatible with j, namely $1..p(j)$. If no, it is an optimal solution among $1..j-1$.

$O(n \log n)$: sort + binary searches; the DP itself is $O(n)$.

WHY NOT GREEDY?

Earliest finish is optimal for the *unweighted* version (L01, L08). With weights: $[0, 10)$ of value 10 vs. $[0, 1), [1, 2)$ of values 1, 1. Earliest finish takes 2, but the optimum is 10. Greedy by value or by value/length also fails (exercise).

MODELLING POINT

Sorting by finish time is what makes "the last interval" a well-defined last decision, and makes the state a single index.

LIS: changing the state changes the complexity

State 1 (L06): $E[i]$ = the LIS length *ending at* i. $E[i] = 1 + \max_{j<i, a_j<a_i} E[j]$: $O(n^2)$.

State 2: after processing $a_1..a_i$, let

$$T[k] = \min\{\text{last element of an increasing subsequence of length } k\}.$$

```
1  LIS(a)
2    T = empty array
3    for x in a
4      k = first index with T[k] ≥ x    ▷ binary search
5      if none: append x to T
6      else:    T[k] = x
7    return length(T)
```

INVARIANT

T is strictly increasing, and $T[k]$ has the meaning above for the processed prefix.

MAINTENANCE

A new x can extend exactly the subsequences whose last element is $< x$. The longest such has length $k-1$, where k is the first index with $T[k] \geq x$. So a length-k subsequence ending in x exists, and $x \leq T[k]$ improves (or keeps) $T[k]$. No other entry can improve. Strictness is kept because $T[k-1] < x \leq T[k]$.

■

$O(n \log n)$. The state changed from "position" to "best tail per length": a **dominance** argument. Only the smallest tail per length matters.

Three ways to make a DP cheaper

Technique	Idea	Example
State reduction	Remove information the future does not need, or merge dominated states	LIS $E[i] \to T[k]$; knapsack Pareto lists (Block B)
Transition optimisation	Compute each $\min$ / $\max$ over predecessors faster, using monotonicity or a data structure	binary search (LIS, weighted intervals); prefix-max with a Fenwick tree; Knuth's optimisation for interval DPs ($O(n^3) \to O(n^2)$); convex-hull trick
Space optimisation	Keep only the part of the table the evaluation order still needs	two rows for LCS/edit distance; one row for knapsack (iterate c downwards); Hirschberg for reconstruction

> Each technique needs its own correctness argument: a monotonicity lemma, a dominance lemma, or a proof that the discarded values are never read again.

Exercise A: model it

1. Weighted intervals: show that greedy by **largest value** and greedy by **largest value per unit length** both fail. (Give explicit instances with costs.)
2. **Russian dolls.** Envelopes (w_i, h_i). One fits inside another iff both dimensions are strictly smaller. Find the longest nesting chain in $O(n \log n)$. (Hint: sort cleverly, then reduce to LIS.)
3. **Counting.** Modify the $O(n^2)$ LIS DP to count *the number* of longest increasing subsequences. What must each state now store?

Pairs · 10 min

BLOCK B · 01:10–02:00

Knapsack and pseudo-polynomial time

Correctness · space · memo vs. tabulation · dominance · encoding

0/1 knapsack: the DP and its correctness

Items $1..n$ with integer weights w_i and values v_i; capacity W.

State: $K[i][c]$ = the maximum value using a subset of items $1..i$ with total weight $\le c$.

$$K[i][c] = \max\big(K[i-1][c],\ v_i + K[i-1][c-w_i] \ (\text{if } w_i \le c)\big), \quad K[0][c] = 0.$$

Time $\Theta(nW)$ · **space** $\Theta(nW) \to \Theta(W)$:

```
for i = 1 to n
   for c = W downto w[i]          ▷ downwards!
      K[c] = max(K[c], K[c − w[i]] + v[i])
```

ASK

Why must c go **downwards** in the 1-D version? What problem do you solve if it goes upwards?

GREEDY FAILS

By value density: $W = 10$; items $(w, v) = (6, 7), (5, 5), (5, 5)$. Densities $1.17, 1, 1$: greedy takes item 1 (value 7) and nothing else fits; the optimum is items 2+3 (value 10). Greedy *is* optimal for the **fractional** knapsack.

Memoization vs. tabulation: an engineering decision

	Memoization (top-down)	Tabulation (bottom-up)
States computed	only those **reachable** from the goal	all states in the table
Evaluation order	implicit (recursion)	must be designed (topological)
Space reduction	hard (needs every cached state)	easy (rows/diagonals)
Overheads	recursion, hash map lookups, stack depth $O(n)$	tight loops; cache-friendly; vectorisable
Knapsack with huge W but few items	visits $\leq \min(nW,\, 2^{n+1})$ states	allocates nW regardless

> Neither choice changes the worst-case bound. They differ in which states are touched, memory, and constants. Choose by the **sparsity** of the reachable states and by memory limits.

Dominance: Pareto sets instead of a W-sized table

Represent the partial solutions over items $1..i$ as (weight, value) pairs. Pair (w, v) **dominates** (w', v') if $w \le w'$ and $v \ge v'$. A dominated pair can never lead to a better final solution: discard it.

```
P0 = {(0,0)}
for i = 1 to n
    Q = P(i−1) ∪ {(w + w_i, v + v_i) : (w,v) ∈ P(i−1), w + w_i ≤ W}
    P(i) = the non-dominated pairs of Q   ▷ merge of 2 sorted lists: O(|Q|)
```

Time $O\left(\sum_i |P_i|\right)$ (Nemhauser–Ullmann 1969). Worst case exponential, but independent of W, and often small on real data.

VALUE-INDEXED DP

If values are small integers: $M[i][v]$ = the minimum weight to achieve value exactly v with items $1..i$. $O(n \cdot \sum v_i)$ time, independent of W. This is the basis of the knapsack **FPTAS** (L14): round the values, then use this DP.

Which one to use depends on which number is small: W, $\sum v_i$, or the Pareto frontier.

Input size and pseudo-polynomial time

DEFINITIONS

An algorithm is **polynomial** if its running time is polynomial in the input *length* (bits). It is **pseudo-polynomial** if it is polynomial in the input length *and the numeric values* of the input, equivalently, polynomial when the numbers are written in **unary**.

- Knapsack input: $O(n \log W + \sum_i (\log w_i + \log v_i))$ bits.
- $O(nW) = O(n \cdot 2^{\log W})$: exponential in the bits of W.
- Polynomial when $W \leq n^c$ (or the numbers are given in unary).

Consequence (L13): knapsack and subset sum are NP-complete, but only **weakly**. Their hardness needs large numbers. **Strongly** NP-hard problems (e.g. TSP, 3-partition) stay hard even with numbers bounded by a polynomial in n.

A PSEUDO-POLYNOMIAL ALGORITHM DOES NOT PROVE P = NP

An $O(nT)$ algorithm for subset sum is exponential on instances with $T \approx 2^n$, which are exactly the hard instances produced by the NP-hardness reduction (L13).

Exercise B: which knapsack algorithm?

Instance	n	W	values	Your method + cost
(a)	200	10^5	up to 10^9	
(b)	200	10^{12}	integers ≤ 100	
(c)	40	10^{15}	up to 10^{15}	
(d)	5000	10^{12}	up to 10^{12}; weights ≈ values ± 1%	

Pairs · 8 min · justify each with the dominant term

BLOCK C · 02:10–02:55

Evidence and problem solving

What the numbers say · midterm-style practice

Experiment: state design and encoding, measured

LIS, random permutation n	$O(n^2)$ ms	$O(n \log n)$ ms	LIS
5,000	30.9	0.1	135
20,000	496.8	0.6	273
50,000	3,090	1.9	432
1,000,000	–	42.0	1,975

Knapsack, $n = 100$, capacity W	bits(W)	DP ms	table
10^3	10	0.1	8 KB
10^5	17	5.7	0.8 MB
10^7	24	1,271	80 MB
10^8	**27**	**15,108**	**800 MB**

`Student_Resources/dp2_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 5 MIN

1. LIS: the $O(n^2)$ column grows ×16 when n grows ×4. Consistent?

2. Knapsack: W ×10 from 10^7 to 10^8, but time ×12. Why not exactly ×10?

3. Add 3 bits to W: what happens to the time? Add 3 items?

Exercise C: midterm-style practice

1. **Optimal BST.** Keys $k_1 < \cdots < k_n$ are searched with frequencies $f_1..f_n$; a search for k_i costs (depth of k_i) + 1. Build the BST minimising the total cost. Give the state, recurrence, base cases, order and complexity, and justify the "choice of root" case.
2. **Coin change (minimum coins)** with coin values $c_1..c_k$ and target T. Give the DP. Is it polynomial? When is greedy optimal? (Give a coin system where it is not.)
3. **Critique.** A student claims: "Subset sum with n numbers each at most n^2 is solvable in polynomial time." True or false? Justify precisely.
4. **Space.** Edit distance for two strings of length 10^6: which quantities can you compute in 16 MB of memory, and how?

Groups · 15 min

Synthesis

1. A better **state** can change the complexity (LIS: $n^2 \to n \log n$ via dominance).
2. Optimise in three ways: state reduction, transition speed-ups (monotonicity, data structures), space reduction. Each needs its own proof.
3. Memoization vs. tabulation: same worst case; different reachable states, memory and constants.
4. Knapsack: $O(nW)$, $O(n \sum v)$, Pareto lists, meet-in-the-middle. Choose by which number is small.
5. **Pseudo-polynomial** = polynomial in the numeric values, exponential in their bit length. That is the gateway to weak vs. strong NP-hardness.

NEXT · L08 (SAT 14 NOV)

Greedy algorithms: exchange and staying-ahead proofs, counterexamples, Huffman coding, matroids.

Midterm (L01–L07, 90 min): working date Sat 14 Nov, in a separate exam slot (L08 is a full lecture); time, room and materials announced in Classroom. **Project M3** (design/methodology) due Sat 14 Nov.